

From Optimal Control to Diffusion Policy

Behavioural Cloning vs. Diffusion Policy on Double-Pendulum Swing-Up & Cube Stacking

Group Project TN2

June 29, 2026

Introduction

Part I — Double-Pendulum Swing-Up

Part II — Robotic Cube Stacking

Conclusion

Introduction

Goal of the project

Question

Compare two **imitation-learning** control methods on the same data.

BC (baseline)

Regress the expert action directly from the state — a single deterministic map.

Diffusion Policy

Generate an action *chunk* from noise by iterative denoising, conditioned on the state.

Two tasks

- Double-pendulum **swing-up** (small, fast, unstable balance task)
- Robotic **cube stacking** — Stack (2 cubes) and StackThree (3 cubes)

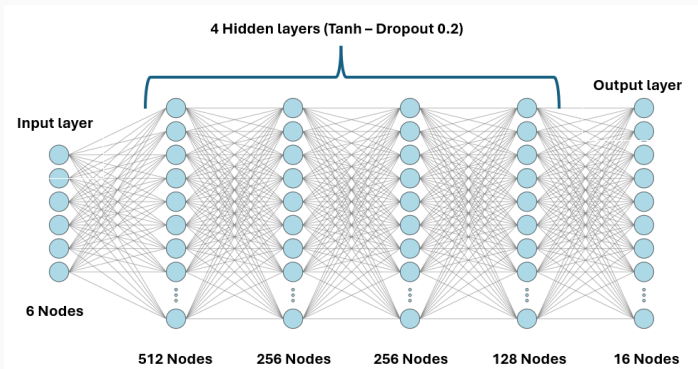
Same demonstrations per task $\Rightarrow$ the comparison reflects the *method*, not the data.

Part I — Double-Pendulum Swing-Up

1. **BC model** — network & training
2. **Diffusion background** — forward / reverse process
3. **Generating the expert dataset** — TrajOpt $\rightarrow$ TVLQR $\rightarrow$ rollout
4. **Diffusion Policy model** — conditioning, network, control tricks
5. **Comparison** — metrics, robustness, ablations

1. Behavioural Cloning - Network Architecture

- Input: feature transform $\phi(s_t) \in \mathbb{R}^6$
- 4 hidden fully-connected layers, tanh, dropout with probability 0.2
- Output 16 $\rightarrow$ reshaped to 8×2 torques

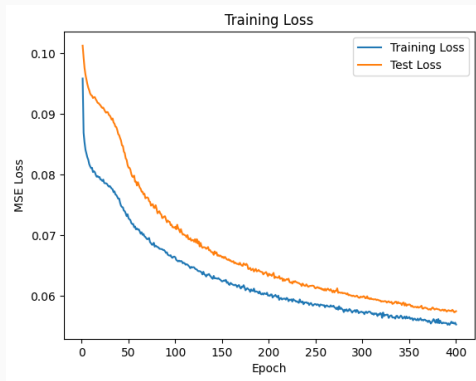


BC Model architecture.

2. BC - Training

Learn $\pi = (a_{t:t+7} \mid s_t)$ — an 8-step action chunk.

- Wrap-safe angular features
 $\phi = [\sin q_1, \cos q_1, \dots, \tilde{q}_2] \in \mathbb{R}^6$
- Min-max normalise to $[-1, 1]$ actions and velocities (stats saved)
- 90/10 train/val
- MSE loss, Adam 10^{-3} , early stop (no overfit)
- 512 batch size
- 400 epochs



Train/test loss — stopped early

2. Diffusion models — background

Forward (noising):

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Reverse (denoising): a network $\epsilon_\theta(x_t, t, c)$ predicts the added noise.

Training (ϵ -MSE)

- Sample t , add noise to get x_t
- Minimise $\|\epsilon_\theta(x_t, t, c) - \epsilon\|_2^2$

Sampling (control time)

- Start from pure noise, iterate $T \rightarrow 1$
- **Deterministic**: drop ancestral noise $\Rightarrow$ posterior mean, low variance

Formulation follows *Understanding Deep Learning* (Prince, 2023).

2. DDPM sampler — double pendulum

Sample an action chunk by **ancestral denoising** (DDPM): start from pure noise $a_T \sim \mathcal{N}(0, I)$ and iterate $t : T \rightarrow 1$ with $T = 20$.

One reverse step

$$a_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \left(a_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(a_t, t, c) \right) + \sqrt{\beta_t} z$$

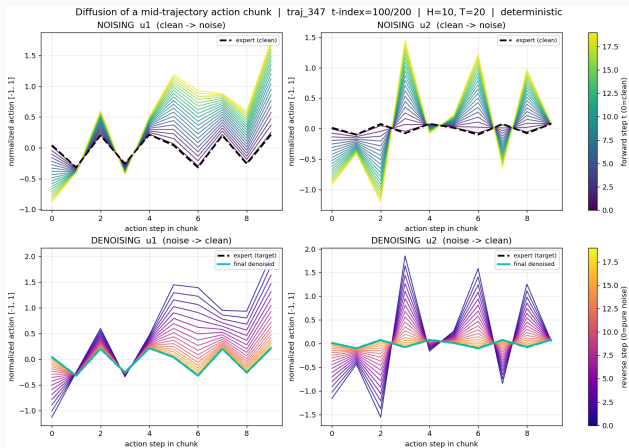
- Linear β -schedule ($10^{-4} \rightarrow 0.02$), EMA 0.999
- *Markovian*: every step removes a little noise, re-adds a little ($z \sim \mathcal{N}(0, I)$)

Control tweak: drop z

Set ancestral noise $z = 0 \Rightarrow$ posterior mean each step. Low action variance is what lets it *hold* the unstable upright — stray torque noise topples the pendulum.

All $T = 20$ steps run every re-plan ($n_{\text{exec}}=1$).

2. DDPM in action — noising & denoising a chunk



One mid-rollout expert chunk ($H=10$, two torques u_1, u_2). **Top:** forward process buries the expert (dashed) in noise; **bottom:** the deterministic reverse process recovers it almost exactly (cyan $\approx$ dashed, reconstruction MSE $\sim 10^{-6}$).

3. Generating the expert dataset

Classical optimal-control pipeline — every sample dynamically valid by construction.

1. **Dynamics model** — planar 2-link EoM, all parameters read from `dp.xml`; smooth tanh friction (differentiable).
2. **Trajectory optimisation** — direct collocation NLP solved with IPOPT (JAX autodiff); swing $[0, 0, 0, 0] \rightarrow [\pi, 0, 0, 0]$.
3. **TVLQR tracking** — $u_t = u_t^* - K_t(x_t - x_t^*)$; nearest-neighbour re-lock for recovery.
4. **Closed-loop rollout** in MuJoCo @ 20 Hz, logging (state, torque).

Dagger-style coverage

Perturb *applied* torque ($\sigma = 0.15\tau_{\max}$), log the *clean* command $\Rightarrow$ teaches recovery from off-nominal states. Keep only successful rollouts (~ 600 trajectories).

4. Diffusion Policy model (Chi et al. 2023)

Denoise an action chunk $a^{1:H}$ ($H = 10$), conditioned on a clean context c .

Representation

- Wrap safe angular features ϕ
- Min-max normalise to $[-1, 1]$ actions and velocities (stats saved)
- $c = \text{last } K=4 \text{ states} + \text{goal} \in \mathbb{R}^{30}$

Network ϵ_θ

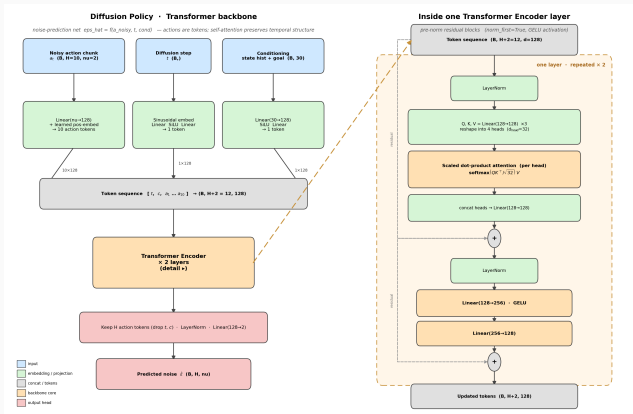
- **Trajectory Transformer** (default) vs. MLP
- Linear β -schedule, $T = 20$ steps, EMA 0.999

What makes the loop stable

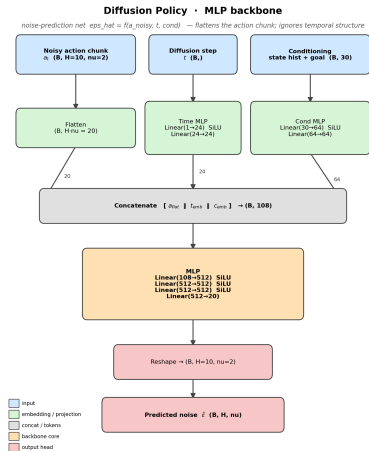
- Deterministic (noise-free) sampling
- **Single-step execution** ($n_{\text{exec}}=1$): re-plan every step

Goal token is effectively constant (one upright goal) $\Rightarrow$ not truly conditionable.

4. Denoiser networks — Transformer vs. MLP

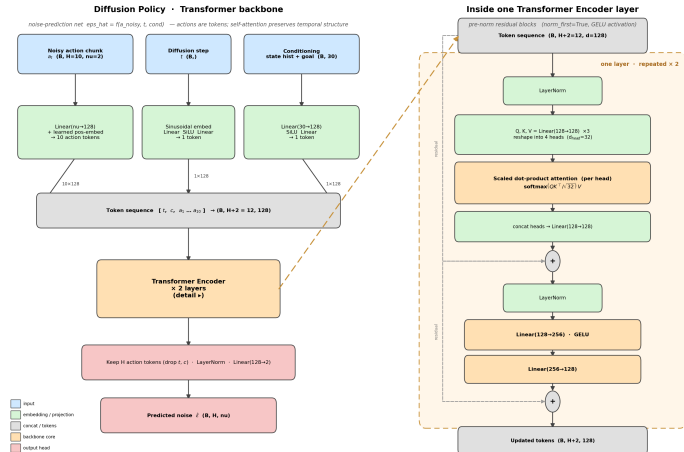


Trajectory **Transformer** denoiser ϵ_{θ} (default).



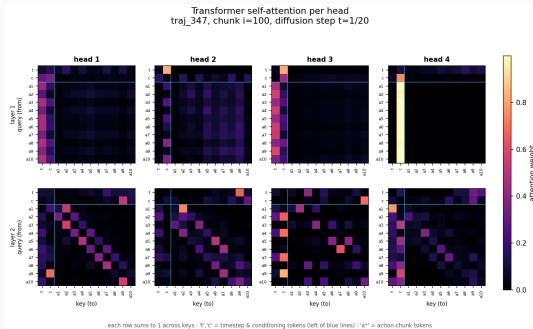
MLP denoiser (ablation baseline).

4. Trajectory Transformer denoiser

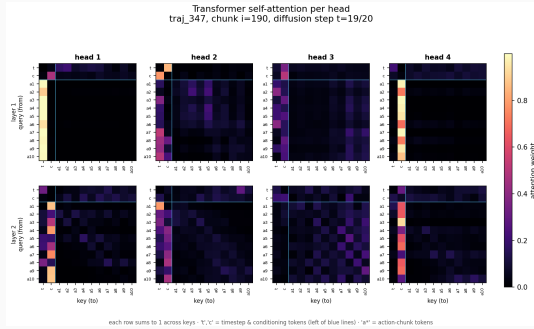


Tokens: diffusion timestep t + conditioning c + action-chunk $a_1 \dots a_{10}$, mixed by self-attention.

Attention heads — early vs. late denoising



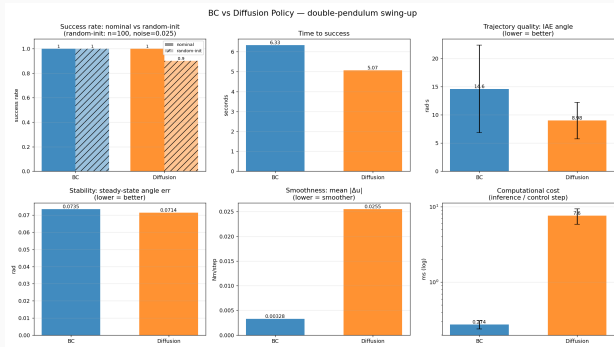
Early denoising ($t=1/20$): action tokens still attend to each other.



Late denoising ($t=19/20$): mass collapses onto the c (conditioning) column.

Rows sum to 1; t, c = timestep & conditioning tokens (left of blue lines), a* = action-chunk tokens.

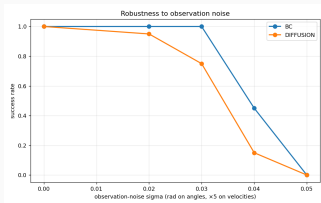
5. Comparison — aggregate metrics



Both swing up: **BC** 100%,
Diffusion 90% (random init).

- **Diffusion** swings up *faster* (5.07 vs 6.33 s) and lower IAE ($\sim 40\%$)
- **BC** holds upright far smoother ($8\times$ less $|\Delta u|$)
- **BC** $\sim 28\times$ cheaper per step (0.27 vs 7.6 ms)

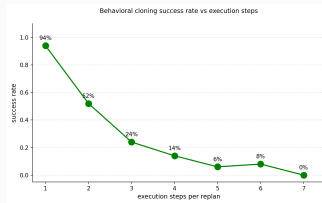
5. Comparison — robustness & ablations



BC degrades more gracefully under observation noise.



Committing to > 1 open-loop action destroys performance.



BC also performs best when open-loop actions are less than 1.

Design choices are not free parameters: transformer backbone (MLP collapses to 2%) and $n_{\text{exec}} = 1$ are the difference between a working and a broken controller.

On the double pendulum, BC is the better controller

Both succeed, but BC holds the upright smoother, is more noise-robust, and $\sim 28\times$ cheaper. The diffusion policy is fragile (needs per-step replanning, chatters near the unstable equilibrium).

Diffusion's edge — multimodal, temporally-correlated actions — does not pay off on a single, unimodal balance task.

Part II — Robotic Cube Stacking

Task & control setup

Franka Emika Panda (7 DoF) in MuJoCo / robosuite, task-space OSC_POSE control.

Action space (7-D)

- Translation $\Delta x, \Delta y, \Delta z$
- Rotation delta (axis-angle)
- Gripper state $\in [-1, 1]$

State space

- Stack: **32-D** (2 cubes)
- StackThree: **48-D** (3 cubes)
- cube poses, gripper-cube vectors, eef pose

Goal: learn $\pi(a_{t:t+H} \mid s_t)$, receding-horizon control (H -step chunks).

- Source: ~ 10 human demos per task $\rightarrow$ MimicGen auto-generates large datasets $(D_0, D_1, \dots)$.
- Four datasets used: `stack_d0/d1`, `stack_three_d0/d1` — 1000 trajectories each.
- HDF5 / robomimic format; only state-relevant keys used (no images).
- A few source expert demonstration were used to generate new and create a complete dataset.

Why it matters

Demonstrations contain *several valid ways* to finish a stack $\Rightarrow$ a multimodal action distribution — the reason where diffusion is expected to perform better.

BC baseline (MLP)

- 5 linear layers ($256 \rightarrow 512$), LayerNorm, ReLU, dropout 0.1
- State $\rightarrow$ 56 values $\rightarrow$ (8, 7) horizon
- MSE loss, Adam 10^{-3} , 50 epochs

Diffusion Policy (Transformer)

- TransformerDenoiser: 4 layers, $d=256$, 4 heads
- State + timestep tokens prepended to $H=8$ action tokens
- ϵ -MSE, AdamW, $K=100$ steps, EMA 0.995

Testing: 50 episodes per model/task, control @ 20 Hz, horizon 500 steps; success = cubes stacked.

Evaluation — metrics & results

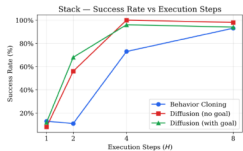
Metrics per planning step: trajectory **jitter** J , **path effort** E , **joint cost** C ; plus **success rate**.

Task	Best success	BC	Diffusion
Stack	($H=8/4$)	93%	98–100%
StackThree	($H=8$)	56%	70%
Replan @ $H=2$ (StackThree)		1%	36%
Jitter ($H=8$, Stack)		0.007	0.022

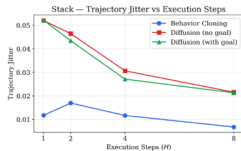
- **Diffusion** stacks more reliably and tolerates more frequent replanning.
- **BC** is smoother (lower jitter) but completes fewer stacks — blurs the multiple valid motions.
- Both need the long $H=8$ horizon; $H=1$ breaks temporal coherence.

Evaluation — cube stacking results

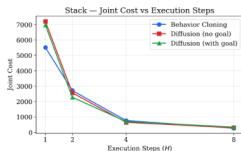
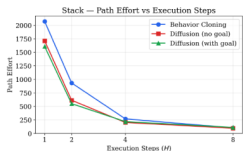
Stack



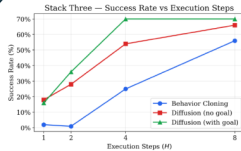
(a) Success rate.



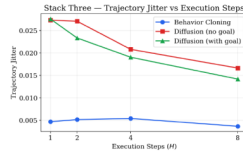
(b) Trajectory jitter.



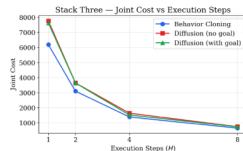
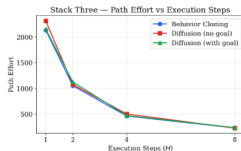
StackThree



(a) Success rate.



(b) Trajectory jitter.



Conclusion

Neither method wins everywhere

The two tasks need *opposite* replanning rates, and each method suits one.

Double pendulum

(fast, unstable, replan every step)

- **BC** wins: smoother, robust, cheap
- Diffusion fragile, chatters

Cube stacking

(multimodal, long horizon)

- **Diffusion** wins: higher success
- BC averages valid motions → fails more

BC: simple, cheap control of small balance tasks.

Diffusion: larger, multimodal manipulation.

Thank you

Questions?